



Summer internship defense • 2026

Building the cognitive games of the Zennyt career platform

Mohamed Slimane • Full Stack Mobile Developer

1 June – 30 August 2026

Supervisors: Mehdi Mtir, Abdelmonem Aisa



The internship at a glance



3

months on the product

1 June – 30 August 2026



14

mini-games

across 9 cognitive families



6

languages

EN · FR · ES · DE · IT · NL



2,372

automated tests passing

843 backend + 1,529 Flutter

Today

1

Context & objectives

2

Architecture & stack

3

My contribution: the games

4

Testing & results

5

Lessons & next steps

One platform, three audiences

Zennyt is a mobile career platform where profiles, job offers, assessments and conversations meet.



Candidates

Build a profile, explore offers, play cognitive games, take technical tests and chat with recruiters.



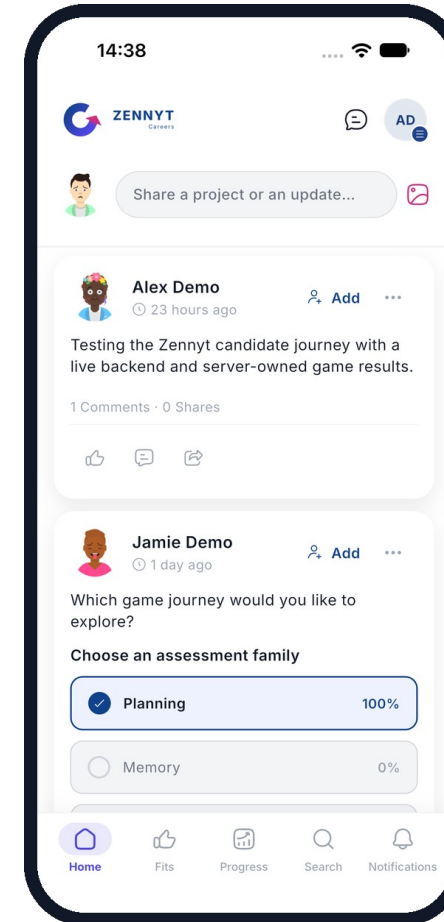
Recruiters

Publish offers, author technical tests and review applicants with a Fit Score.

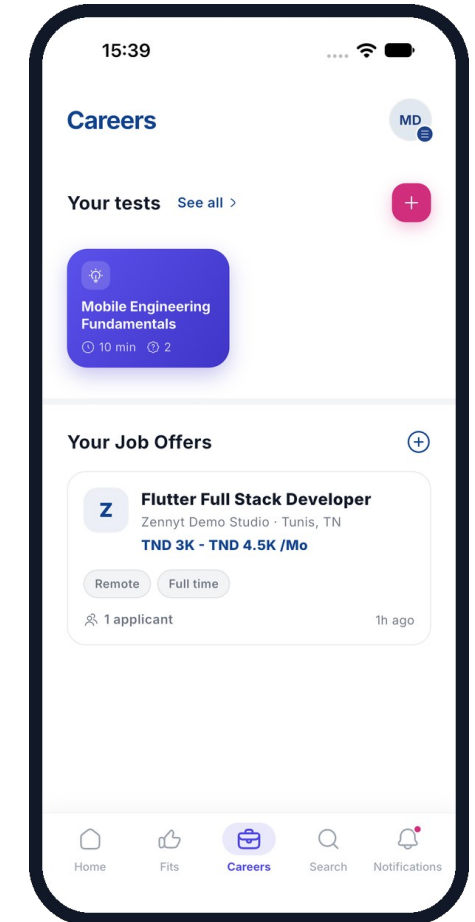


Staff

Game Studio and Platform Console manage assessment content and platform operations.



Candidate · Home feed



Recruiter · Careers dashboard

The challenge: trustworthy game results

Why it is hard



The phone captures every action and timing precisely



The backend validates the session and applies the scoring rule



The result stays consistent across games, languages and screens

Objectives

01

Implement interactive
Flutter game journeys

02

Connect raw
measurements to server-
side scoring

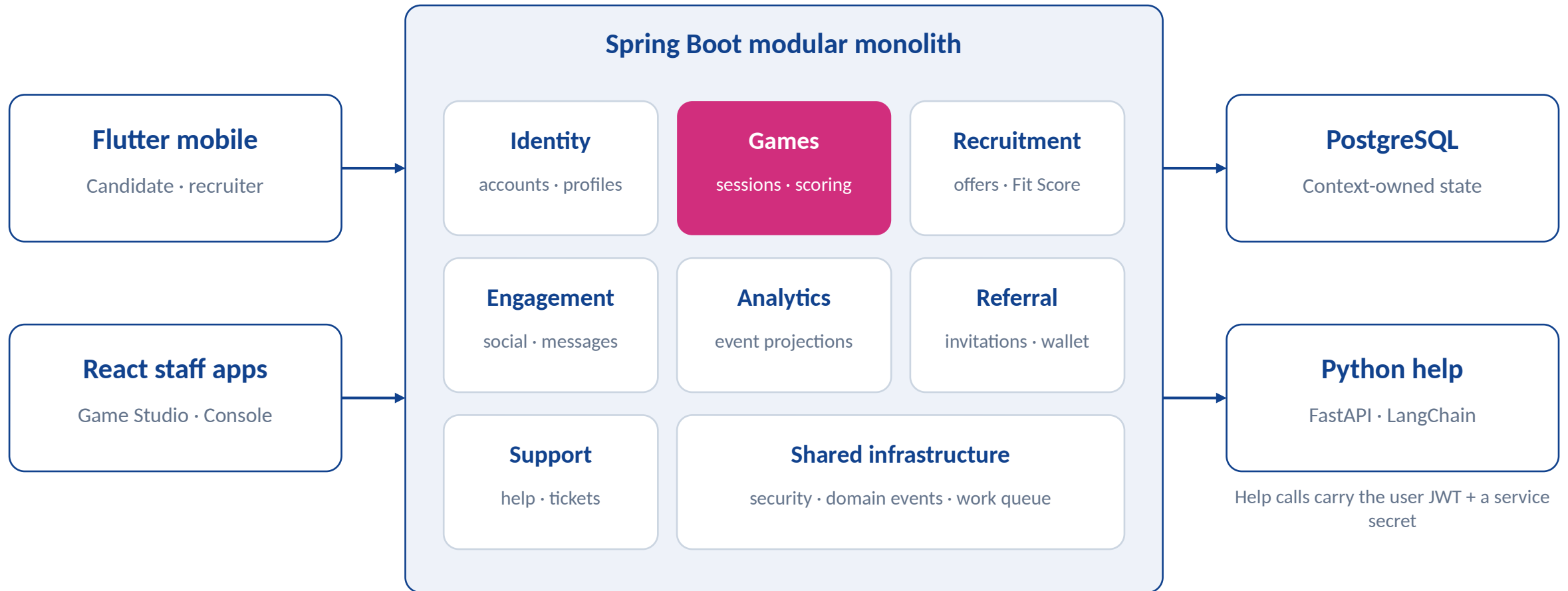
03

Manage sessions, tutorials
and lifecycle states

04

Reusable result screens in
six languages

A modular monolith behind two clients



Each context: API → application → pure domain → infrastructure, enforced by ArchUnit. Contexts talk through domain events; contracts start in OpenAPI.

Technology stack by layer

Mobile



- Riverpod state
- go_router navigation
- Flame game runtime
- Dio + STOMP client
- Secure storage, Hive

Backend



- Spring Boot modular monolith
- Spring Data JPA
- Spring Security, OAuth2
- OpenAPI Generator

Administration



- TanStack Start
- Game Studio
- Platform Console

Help service



- LangChain
- Chroma retrieval
- Authenticated AI gateway

Quality & runtime JUnit · ArchUnit · Flutter test · OpenAPI contract-parity tests · Compose · Maven

My focus: 9 families, 14 mini-games



Planifik

Optimal Path • Task Scheduling •
Predictive Puzzle



Move Fast

Move Fast Core



Memory Quest

I Investigate



Decision

I Decide



Emotional Regulation

Emotional Radar • Reflective
Pause • Strategic Choices



Continuous Attention

I Continue



Visuomotor Coordination

I Coordinate



Visuospatial Memory

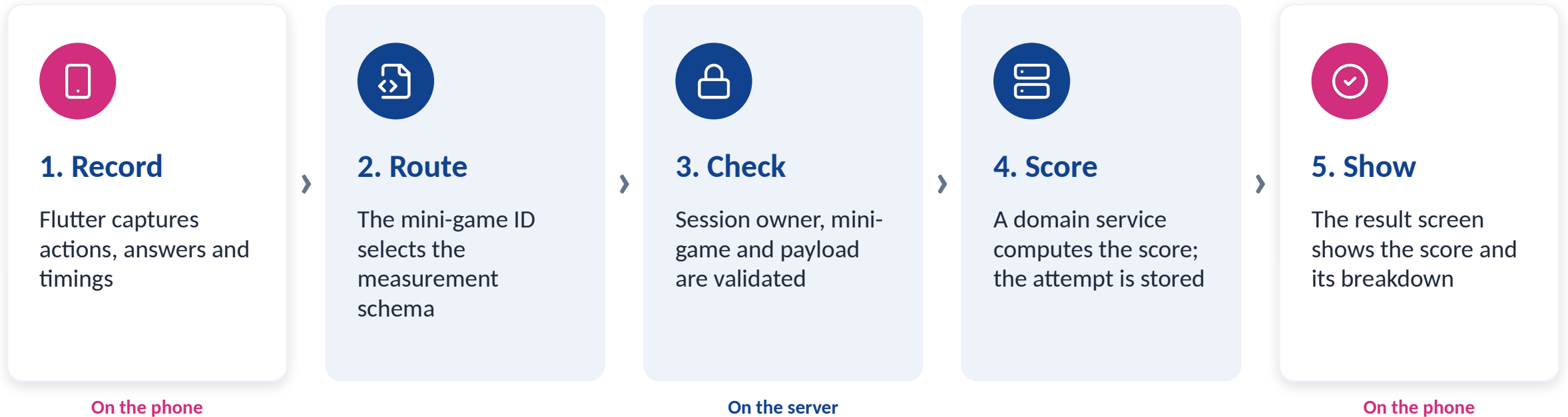
I Place



Decision Behavioral

BART • IST

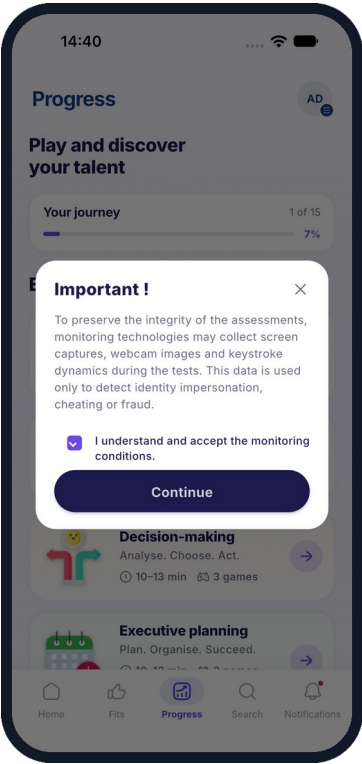
The client measures, the server decides



The client never chooses an authoritative score

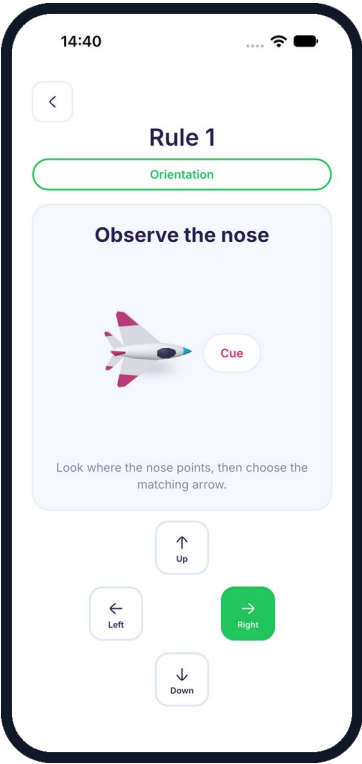
Invalid metrics, foreign sessions, incompatible mini-games and duplicate results are rejected. Technically invalid runs are kept for audit, never counted.

One shared journey for every game



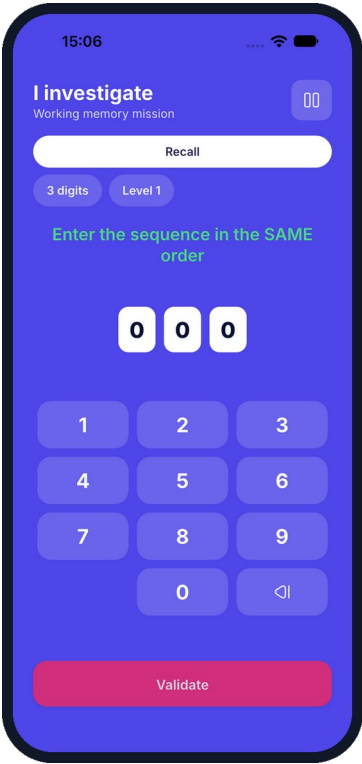
1 Consent

Monitoring conditions accepted



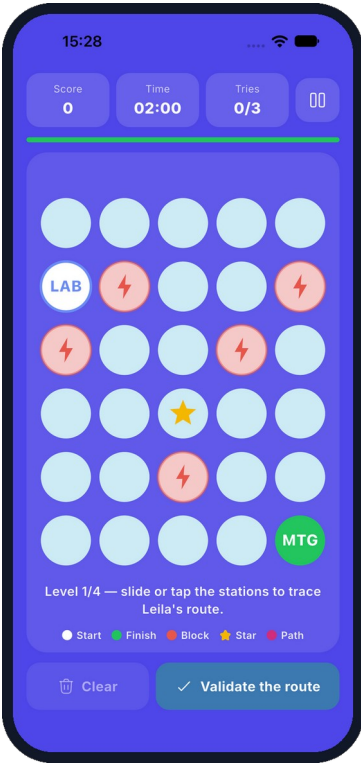
2 Instructions

Move Fast rule with a cue



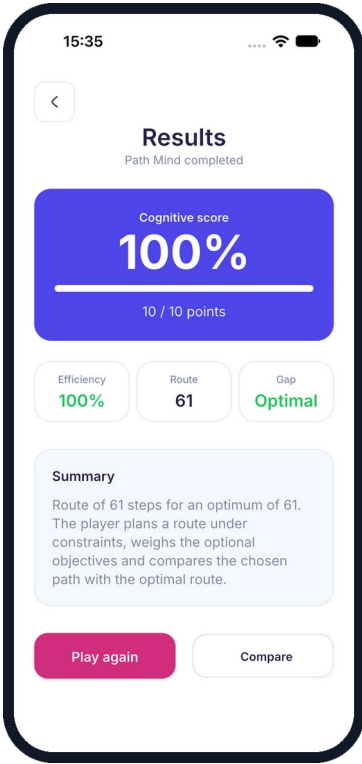
3 Gameplay

Memory Quest digit recall



4 Planning

Optimal Path route board



5 Results

Server score: 10 / 10

Replay: the server rebuilds the run



BART

Pumps and cash-outs per balloon → the server reconstructs explosions and earnings



IST

Sampled boxes and choices → the server rebuilds what was visible at decision time

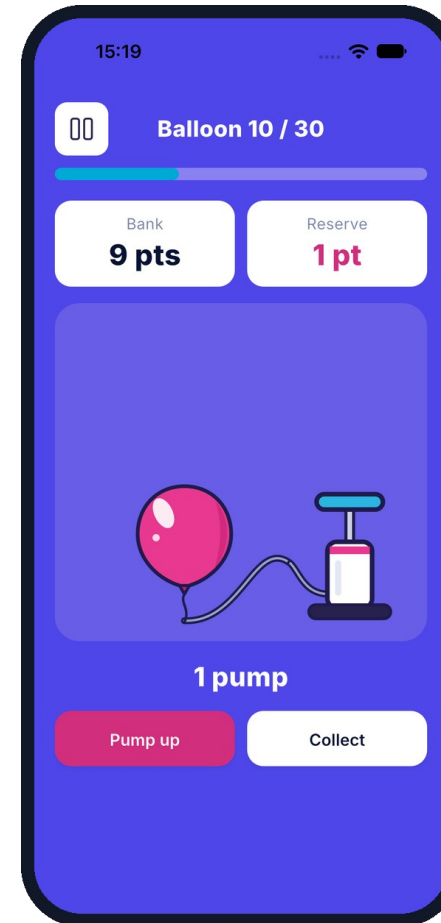


I Place

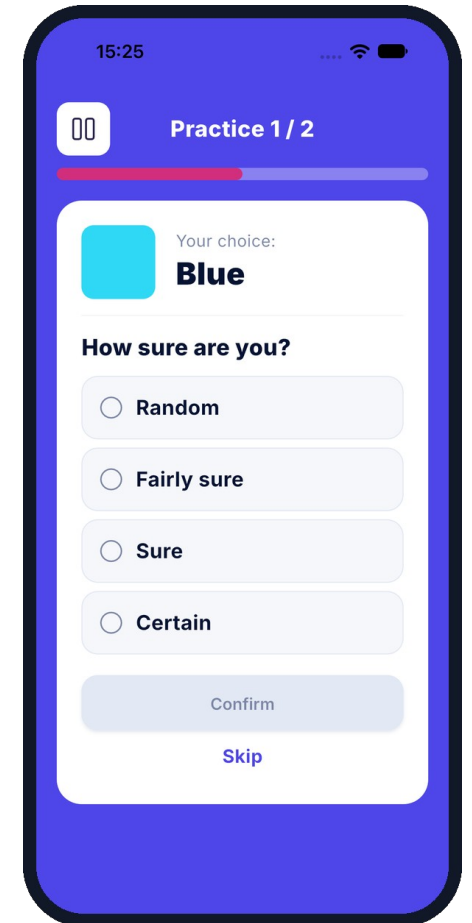
Placement actions are replayed against the controlled layout

Controlled content

Runtime snapshots freeze the published settings when a session starts; assigned forms bind scenario games to a controlled bank.

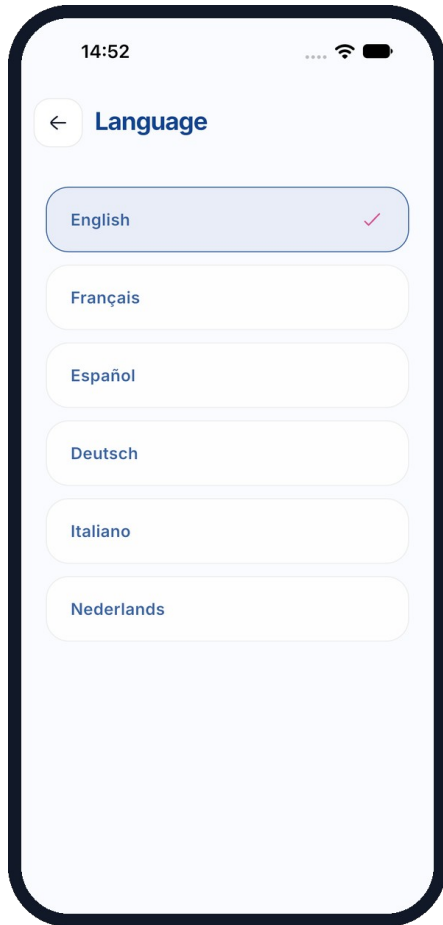


BART • balloon 10 of 30

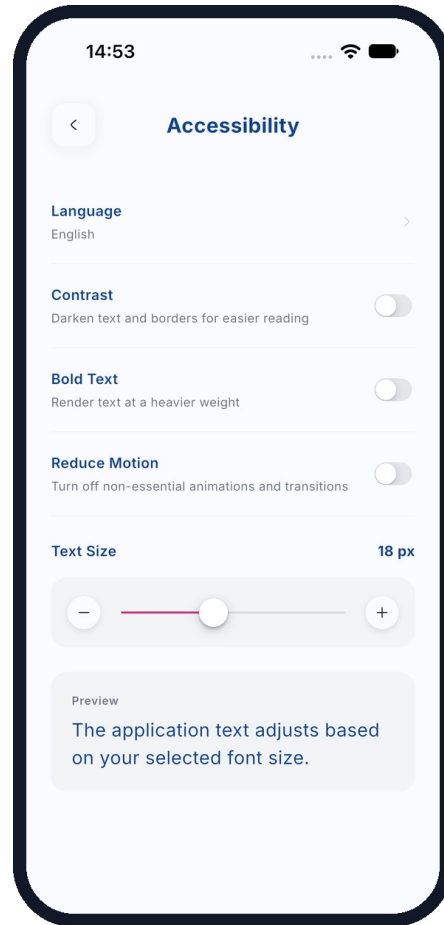


IST • colour choice + confidence

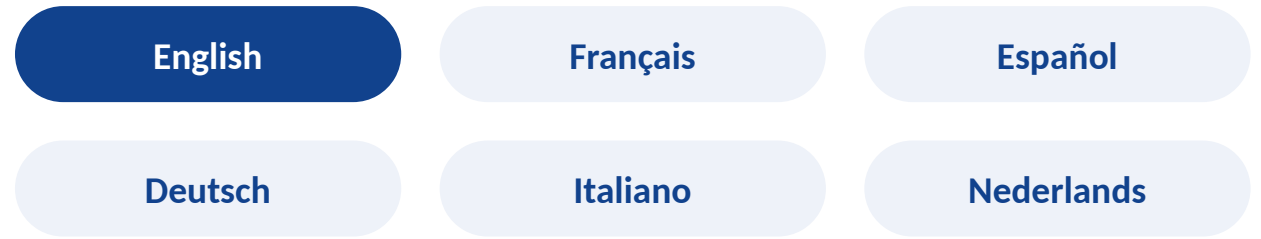
Six languages, adjustable reading



Language picker



Accessibility settings



Localization keys everywhere

Instructions, controls, errors and accessibility labels; ICU plurals for counts.



Layouts that absorb long strings

Shared game layouts keep German and Dutch readable on small screens.



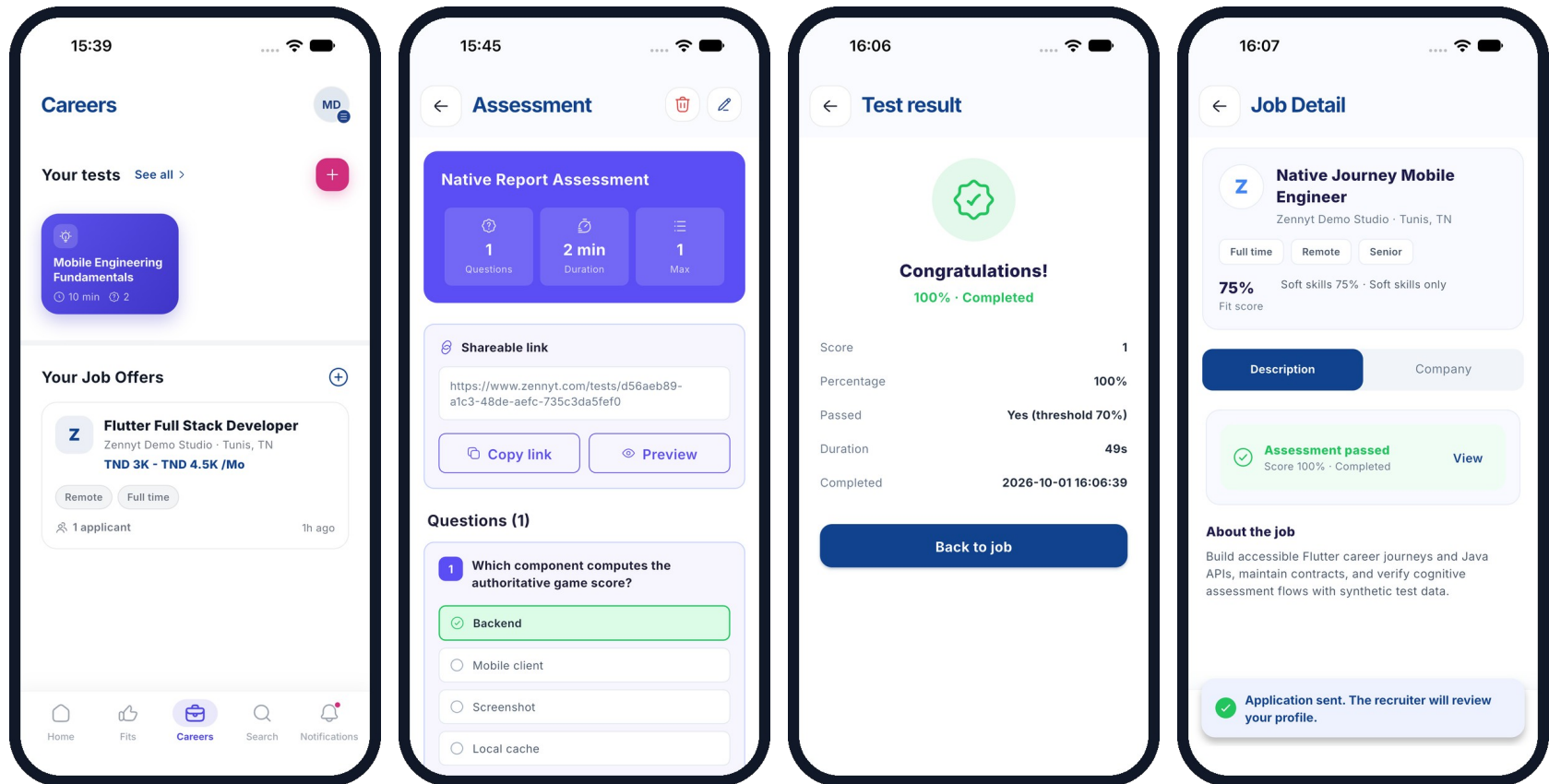
Reading preferences

Text size, contrast, bold text and reduced motion.

From recruiter test to application

Beyond the games, I tested the full recruiter-to-candidate path on a real iPhone build.

- 1 **Recruiter** creates a test and publishes an offer
- 2 **Candidate** accepts consent and answers shuffled questions
- 3 **Server** grades the attempt: 100%
- 4 **Candidate** applies with the passed-test banner



Careers dashboard Test and answer options Server result: 100% Application sent

Automated suites are green

843

backend Maven tests passed

0 failed • 41 skipped

1,529

Flutter tests passed

0 failed • 0 skipped

218 / 221

API checks passed

2 failed • 1 blocked



Test environment

Spring Boot dev backend and local PostgreSQL; Flutter iPhone app driven by XCTest with game mocks off; synthetic accounts, offers and posts.



Not yet covered

Email and OTP delivery, push, call media, real payments and payouts, multi-device. The AI help gateway returned HTTP 503.

Walkthrough: 1 passed, 6 partial

Use case	Workflow	Status	Principal observation
UC-01	Cognitive assessment	Partial	Planning result saved natively; memory payload failed
UC-02	Offer management	Partial	Offer published; optional test picker failed
UC-03	Technical assessment	Passed	Native completion; result read back by both parties
UC-04	Communication	Partial	Message persisted; call media and delivery unverified
UC-05	Community	Partial	Post and comment persisted; self-friend action visible
UC-06	Preferences and help	Partial	Preferences persisted; AI service unavailable
UC-07	Referrals and plans	Partial	Local state verified; real payouts not exercised

Partial: the core path persisted, but a dependent step or external service failed or was not verified.

Five defects found and documented

Area	What happened	Fix direction
 Memory Quest	Client sends mode instead of memoryQuestMode; the digit exercise keeps the default mode, so the result is not saved	Align the payload with the contract; add a parity test
 Decision forms	Another candidate could read a player's assigned form	Add an ownership check on the read endpoint
 Assessment picker	Offer form resolves /assessments/pick as an assessment ID → HTTP 400	Match the static route before /{id}
 Localization	Notification content appeared in French during an English session	Render notifications in the session locale
 Community	An author's own post offered to add the author as a friend	Hide relationship actions on own content

What I take away

“Treat a game as a complete feature: instructions, interaction, measurement, validation, scoring and feedback must stay consistent from Flutter to the backend.”

Skills developed

Mobile ↔ backend integration

Game protocols as typed payloads

Async state across screens

Next steps

1

Fix the five defects, starting with the ownership check

2

Integration-test external services: email/OTP, push, calls, payments

3

Bring the AI help service online and re-run the walkthrough



Thank you

Questions and discussion

Mohamed Slimane · Full Stack Mobile Developer · Zenny